A Project Report on

DEEP EMBEDDED CLUSTERING-BASED SEMANTIC USER GROUPING FOR UAV-ASSISTED DISASTER COMMUNICATION

submitted in partial fulfilment of the requirements for the course

MACHINE LEARNING
25SC2107E

in

A.Y.: 2026-27

by

CHANDRAKANTH	2520040098
PAVAN KARTHIK	2520040071
KARTHIK	2520040047

Under the Esteemed guidance of

Mr. N Lava Prasad

Assistant Professor,

Department of ECE



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF **ELECTRONICS & COMMUNICATION ENGINEERING**

Abstract:

Communication networks can become unavailable or overloaded during disasters such as floods and earthquakes. Victims may need different kinds of support, including general information, medical assistance, location sharing, or rescue coordination. Organizing requests by communication need can help a future unmanned aerial vehicle (UAV)-assisted communication system manage messages more effectively. This project presents a machine learning workflow for Deep Embedded Clustering-Based Semantic User Grouping for UAV-Assisted Disaster Communication.

Repository file data/disaster_users.csv is a synthetic dataset of 1,000 users, transformed into ten inputs: six standardized numerical features (x/y location, channel condition, emergency priority, delay requirement, and semantic similarity) plus four one-hot requested-data categories. The user ID is excluded. K-Means and DBSCAN are baselines for Deep Embedded Clustering (DEC). DEC encodes the inputs through 32- and 16-unit layers into five latent dimensions, initializes four centers using K-Means, and refines assignments with a target distribution and Kullback-Leibler divergence.

Saved outputs summarize four interpreted groups: General, Medical, Location, and Rescue. Medical and Rescue have higher average priority and shorter delay tolerance than General requests. The repository's three internal clustering scores are near-identical and do not demonstrate a meaningful DEC advantage. Because the inputs are synthetic, these profiles are not validated representations of people affected by disasters. Real-data testing and evaluation in a UAV network simulator remain future work.

Signature of the Supervisor

Course Instructor

TABLE OF CONTENTS

Page numbers follow the page footer in this PDF.

Section	Page
Abstract	2
Chapter 1 - Introduction	5
Chapter 2 - Background and related concepts	7
Chapter 3 - System model and data preparation	9
Chapter 4 - Proposed methodology	11
Chapter 5 - Implementation and execution pipeline	13
Chapter 6 - Experimental results and discussion	15
Chapter 6 - Visual outputs and result analysis	18
Chapter 7 - Conclusion and future work	23
References	24
Appendix A - Project summary	24
Appendix B - Abbreviations	24

CHAPTER 1: INTRODUCTION

1.1 Background

Disasters can damage cellular towers, power systems, and communication links. People affected by the same event may have very different needs: one person may request general information, another may need medical assistance, and another may need to share a location or request rescue. When a communication system has limited capacity, treating every request in exactly the same way may not be the most useful strategy.

Unmanned aerial vehicles (UAVs), often called drones, are potential platforms for temporary communication support because they can be moved to areas where ground infrastructure is unavailable. Before communication resources can be allocated, incoming users or requests can be organized into groups based on characteristics such as location, channel quality, emergency priority, delay tolerance, and the type of information requested.

1.2 Problem Statement

Normal communication networks may fail or become overloaded during floods, earthquakes, and other disasters. The project addresses the problem of organizing 1,000 users into meaningful groups based on communication-related features, so that these groups can be used as input to a future UAV-assisted communication system.

1.3 Objectives

Prepare user data for unsupervised machine learning.

Create a ten-feature input matrix from numerical and categorical information.

Implement and compare K-Means, DBSCAN, and Deep Embedded Clustering (DEC).

Interpret four resulting groups using their average characteristics.

Provide a workflow for assigning a group to a new user and viewing results through a Streamlit dashboard.

1.4 Scope

The scope is limited to data preparation, clustering, group interpretation, comparison metrics, and a simple dashboard. The project does not claim that a live drone network is deployed, nor that the clusters directly guarantee faster rescue or improved survival outcomes. Such claims would require separate field evaluation.

CHAPTER 1: INTRODUCTION (CONTINUED)

1.5 Motivation

During an emergency, message urgency and message type can be important alongside network conditions. A structured grouping step can help a later system distinguish routine information from urgent medical or rescue-related requests. Machine learning is useful here because the data may not include reliable labels in advance, and patterns may emerge from the combined feature space.

1.6 Expected Outcomes

A prepared dataset with 1,000 rows and ten model input features.

Cluster assignments from three unsupervised learning methods.

Four interpreted groups with human-readable names.

Summary metrics for comparing cluster structure.

A reproducible project pipeline and interactive dashboard for exploring the results.

1.7 Report Organization

Chapter 2 introduces the relevant machine learning concepts. Chapter 3 explains the system model and feature preparation. Chapter 4 describes the clustering methods, with emphasis on DEC. Chapter 5 outlines the software workflow. Chapter 6 presents the reported metrics and cluster interpretations. Chapter 7 summarizes the project and identifies future improvements.

CHAPTER 2: BACKGROUND AND RELATED CONCEPTS

2.1 Unsupervised Machine Learning

Unsupervised learning identifies structure in data without requiring a target label for every example. Clustering is a common unsupervised task: it assigns similar observations to the same group according to a distance or similarity measure. In this project, user records are grouped using communication features rather than known class labels.

2.2 K-Means Clustering

K-Means partitions observations into a chosen number of clusters. It repeatedly assigns each point to the nearest centroid and updates each centroid to the mean of its assigned points. The project uses k = 4 to produce four groups. K-Means is straightforward and useful as a baseline, but it is sensitive to feature scaling, initial centroid placement, and cluster shapes.

2.3 DBSCAN

DBSCAN forms clusters from dense regions and can label isolated records as noise. Repository code searches eps = {0.5, 0.7, 0.9, 1.1, 1.3, 1.5} and min_samples = {5, 10, 15}, selecting the candidate with the highest silhouette among non-noise points. The saved run reports four clusters and zero noise; the winning parameter pair is not recorded in its metrics CSV.

2.4 Deep Embedded Clustering

Deep Embedded Clustering combines a neural-network representation with clustering refinement. An autoencoder learns a compact representation of input features. Cluster centres are initialized in that representation, and soft cluster assignments are iteratively refined. The aim is to learn features that make cluster structure clearer than it may be in the original input space.

2.5 Evaluation Metrics

Silhouette score: measures how close an observation is to its own cluster compared with other clusters; higher is generally better.

Davies–Bouldin index: compares within-cluster scatter with separation between clusters; lower is generally better.

Calinski–Harabasz score: compares between-cluster dispersion with within-cluster dispersion; higher is generally better.

2.6 Why Feature Preparation Matters

The input features use different units and meanings. Coordinates, delay values, and priority values may have different numerical ranges. Standardization reduces scale differences among numerical features so that a feature with a large numeric range does not dominate distance calculations merely because of its units.

Requested data type is categorical, not a continuous measurement. One-hot encoding converts the four categories—General, Medical, Location, and Rescue—into separate binary columns. The model can then use these columns as part of the numerical input matrix.

2.7 Semantic Grouping for Emergency Requests

In this project, the word semantic refers to the meaning or category of a user's communication request, combined with the user's operational characteristics. For example, a Medical request with high emergency priority and a short delay requirement may be interpreted differently from a General request with a more flexible delay. The clustering model itself produces numeric cluster IDs; human-readable names are assigned after studying each cluster's average characteristics.

2.8 Role of UAV-Assisted Communication

The intended downstream use is to supply grouped user information to a future drone communication or network-allocation system. A UAV system could potentially use group labels alongside location and urgency information when planning service. However, this report treats that integration as future work, not as a feature already validated in a live network.

2.9 Summary

The project combines unsupervised clustering, numerical preprocessing, categorical encoding, and representation learning. K-Means and DBSCAN provide comparison baselines, while DEC is the proposed approach. A meaningful comparison must consider both quantitative scores and whether the resulting groups are interpretable for the intended emergency-communication use.

CHAPTER 3: SYSTEM MODEL AND DATA PREPARATION

3.1 System Overview

The system begins with raw user records and prepares a fixed-size feature matrix. The matrix is processed by baseline and proposed clustering methods. Their outputs are analyzed using clustering metrics and cluster-level averages. Finally, the system supports testing a new user and viewing results through a dashboard.

Stage	Input	Operation	Output
1. Data ingestion	1,000 user records	Load and inspect raw columns	Raw dataset
2. Preprocessing	Raw dataset	Drop identifier, scale numbers, encode category	1,000 × 10 matrix
3. Clustering	Processed features	Run K-Means, DBSCAN, and DEC	Cluster labels
4. Analysis	Labels and features	Calculate metrics and profile groups	Scores and group descriptions
5. Presentation	Results and new record	Dashboard and prediction workflow	Visual summary and predicted group

3.2 Dataset Description

The project review specifies 1,000 users and eight raw columns. The user_id column is an identifier and is removed from the machine learning input because it does not describe communication need. The remaining information is transformed into ten features.

Feature / Field	Type	Preparation or use
user_id	Identifier	Removed from model input
x_coordinate, y_coordinate	Numerical	Standardized
channel_condition	Numerical	Standardized
emergency_priority	Numerical	Standardized
delay_requirement	Numerical	Standardized
semantic_similarity	Numerical	Standardized
requested_data_type	Categorical	One-hot encoded into four columns

3.3 Feature Matrix

Six numerical features are scaled to mean 0 and standard deviation 1. The requested_data_type category is represented using four one-hot columns. Therefore, the final input contains 6 + 4 = 10 features. For 1,000 users, the processed matrix has 1,000 rows and 10 columns.

3.4 Feature Meaning

Feature	Interpretation
x_coordinate, y_coordinate	User's position represented in a two-dimensional map coordinate system
channel_condition	Quality or clarity of the communication channel
emergency_priority	Urgency level associated with the user's situation
delay_requirement	Time limit or acceptable delay for the requested communication
semantic_similarity	Similarity of the request to a topic or semantic representation
General / Medical / Location / Rescue	One-hot columns identifying the requested data category

3.5 Preprocessing Procedure

Load the raw user data and inspect the available columns.

Remove user_id from the features used to train clustering models.

Separate the six numerical features from the requested-data category.

Standardize the numerical values so that each feature has a comparable scale.

One-hot encode requested_data_type into General, Medical, Location, and Rescue columns.

Combine the scaled numerical columns and encoded category columns in a consistent order.

Verify that the resulting feature matrix contains ten features for each of the 1,000 users.

3.6 Data Quality Considerations

The repository labels data/disaster_users.csv as 1,000 synthetic user records, not observations collected from people in a live disaster. The report does not document the generation distributions. Before operational use, check missing values, invalid coordinates, category labels, and out-of-range priority or delay values; document data-generation assumptions and limits.

3.7 Privacy and Responsible Use

Location and emergency information can be sensitive. A real deployment should limit access to personal data, remove unnecessary identifiers, protect data in transit and at rest, and define how long records are retained. Cluster assignments should support decision-making rather than replace emergency personnel or determine access to lifesaving assistance without safeguards.

CHAPTER 4: PROPOSED METHODOLOGY

4.1 Overall Method

The proposed method is Deep Embedded Clustering (DEC). The DEC workflow learns a compact representation of the ten input features and then refines cluster assignments in that representation. The project also runs K-Means and DBSCAN so the proposed method can be compared against conventional clustering baselines.

4.2 Autoencoder Architecture

The architecture described in the project review takes ten processed features as input. The encoder uses a layer with 32 neurons followed by a layer with 16 neurons and maps the input to a five-dimensional latent representation. The latent representation is a compressed description of each user's communication characteristics.

Layer	Size	Purpose
Input	10 features	Accept prepared user data
Encoder layer 1	32 neurons	Learn intermediate feature combinations
Encoder layer 2	16 neurons	Further compress representation
Latent representation	5 features	Compact representation used for clustering

4.3 DEC Clustering and Refinement

Train the autoencoder to learn a compact representation of the input features.

Use K-Means to initialize cluster centres in the five-dimensional latent space.

Calculate soft assignments that represent how likely each user is to belong to each cluster.

Construct a target distribution that strengthens high-confidence assignments.

Refine the assignments by minimizing the Kullback–Leibler (KL) divergence between the target distribution and current soft assignments.

Repeat refinement until the training procedure reaches its stopping condition.

4.4 Baseline Configuration

Method	Configuration in saved code/results	Purpose
K-Means	k = 4	Ten-feature baseline
DBSCAN	eps = 0.5, 0.7, 0.9, 1.1, 1.3, 1.5; min_samples = 5, 10, 15. Saved output: 4 clusters, 0 noise; winning pair not stored.	Grid-searched density baseline
DEC	Encoder: 10 -> 32 -> 16 -> 5; four output groups	Learned representation and refinement

CHAPTER 4: PROPOSED METHODOLOGY (CONTINUED)

4.5 Human Interpretation of Cluster IDs

Clustering algorithms generally return numeric labels such as 0, 1, 2, and 3. These numbers do not have a built-in meaning or ranking. After clustering, the project studies average requested data type, emergency priority, and allowed delay for each group. Human-readable names are then assigned to make the output easier to interpret.

4.6 Cluster Naming Logic

General: predominantly general-information requests, with low average emergency priority and flexible delay.

Medical: predominantly medical emergency requests, with high average priority and a very short allowed delay.

Location: predominantly location-sharing requests, with medium average priority and delay.

Rescue: predominantly rescue-operation requests, with high average priority and urgent delay.

4.7 End-to-End Workflow

The overall pipeline is: raw data → feature preparation → baseline clustering and DEC → model evaluation → cluster profiling → result reports and charts → new-user prediction → Streamlit dashboard. The run_all.py script is described in the review as the all-in-one entry point for executing the complete workflow.

4.8 Limitations of the Method

The quality of clustering depends on the representativeness of the data, the choice of features, scaling, parameter settings, and model initialization. A compact latent representation may help reveal structure, but it does not automatically guarantee better clusters. The reported scores are very similar across the tested methods, so the current results should be treated as an initial comparison rather than evidence that DEC is superior.

CHAPTER 5: IMPLEMENTATION AND EXECUTION PIPELINE

5.1 Technologies Used

Technology	Role in the project
Python 3.10	Main programming environment
TensorFlow / Keras	Autoencoder and DEC neural-network implementation
Scikit-learn	Preprocessing utilities and baseline clustering models
Streamlit	Interactive web dashboard
Python scripts and data folders	Modular execution, inputs, results, and reporting

5.2 Modular Structure

The project review describes eleven modular Python scripts under a src/ folder, alongside data/ and results/ folders, a Streamlit application, setup instructions, and run_all.py. The following logical modules summarize the pipeline responsibilities; exact filenames beyond those stated in the review should be verified against the repository.

Module / stage	Responsibility
Data inspection and preparation	Inspect raw records and create the ten-feature matrix
Baseline models	Run K-Means and DBSCAN
DEC model	Train the autoencoder and refine cluster assignments
Analysis	Calculate comparison scores and study group characteristics
Reporting	Generate charts and report outputs
New-user testing	Predict a cluster for a new record and provide confidence information
Dashboard	Present interactive results through Streamlit
run_all.py	Run the full pipeline from one entry point

5.3 Execution Sequence

Install Python 3.10 and the required packages listed in the repository setup instructions.

Place the input dataset in the expected data/ location.

Run the data preparation stage and verify the 1,000 × 10 processed matrix.

Execute the baseline clustering models.

Train and run the DEC pipeline.

Generate model metrics and cluster profile summaries.

Launch the Streamlit application using the repository's documented command.

Test the new-user prediction workflow with a properly prepared input record.

CHAPTER 5: IMPLEMENTATION AND EXECUTION PIPELINE (CONTINUED)

5.4 Dashboard and New-User Prediction

The Streamlit dashboard is intended to provide an accessible view of the model outputs. It can present group descriptions, model comparison results, and a workflow for testing a new user. New records must undergo the same feature preparation as the training data: the numerical columns must use the same scaling parameters and the requested-data category must use the same one-hot column order.

5.5 Reproducibility

Record the Python and package versions used for each run.

Keep the preprocessing configuration consistent between training and prediction.

Save the random seed and model parameters when applicable.

Store model outputs and evaluation metrics in the results/ directory.

Document the dataset source, synthetic or real status, and any data cleaning performed.

5.6 Implementation Notes

The supplied review material confirms the broad software workflow and names the principal technologies, but it does not provide complete package versions, hardware specifications, exact training epochs, optimizer settings, or a full source listing. These implementation details should be copied from the actual repository or experiment logs if required for a formal reproducibility audit.

5.7 Repository

The project review identifies the source repository as:

https://github.com/chandu235363/semantic-uav-disaster-communication

CHAPTER 6: EXPERIMENTAL RESULTS AND DISCUSSION

6.1 Model Comparison

The repository's saved model_comparison.csv reports four clusters for each method. The table reproduces its scores to four decimals; all three methods were evaluated on the same synthetic dataset.

Method	Parameters	Clusters	Silhouette ↑	Davies–Bouldin ↓	Calinski–Harabasz ↑
K-Means baseline	k = 4	4	0.6320	0.5501	2630.6143
DBSCAN baseline	Grid search	4	0.6320	0.5501	2630.6143
DEC proposed	10 → 32 → 16 → 5	4	0.6320	0.5501	2630.6144

6.2 Interpretation

A silhouette score of 0.6320 indicates a reasonably separated clustering structure under the metric's assumptions. A Davies–Bouldin value of 0.5501 reflects the reported balance of within-cluster scatter and between-cluster separation, with lower values preferred. The Calinski–Harabasz values are also very close. Since all reported metrics are effectively the same, the current experiment does not demonstrate a significant performance advantage for DEC over the baselines.

6.3 Important Validation Point

The repository's comparison and per-model CSV files agree to the displayed precision. DBSCAN reports four clusters and zero noise points. Its code searches a parameter grid, but the saved metrics do not identify the winning eps/min_samples pair. The charts below use these saved values; they are not a new recomputation from the labels and feature matrix.

6.4 DEC User Groups

Cluster ID	Assigned name	Main data type	Average emergency priority	Average allowed delay
0	General	General information	1.21 (Low)	5.95 seconds (Flexible)
1	Medical	Medical emergency	2.81 (High)	0.98 seconds (Urgent)
2	Location	Location sharing	2.02 (Medium)	3.00 seconds (Medium)
3	Rescue	Rescue operation	2.81 (High)	1.54 seconds (Urgent)

6.5 Group-by-Group Discussion

Cluster 0 — General

This group is associated mainly with general information. Its reported average emergency priority is 1.21, and its average allowed delay is 5.95 seconds. This profile suggests that, on average, the requests in this group can tolerate more delay than the Medical and Rescue groups.

Cluster 1 — Medical

This group is associated with medical emergencies. Its reported average priority is 2.81 and average allowed delay is 0.98 seconds. These values describe a high-priority, time-sensitive group. Any real system should validate these interpretations with emergency-response experts.

Cluster 2 — Location

This group is associated with location sharing. Its reported average priority is 2.02 and average allowed delay is 3.00 seconds, both described as medium in the review. Location messages can support coordination, but the importance of a particular message depends on context.

Cluster 3 — Rescue

This group is associated with rescue operations. Its reported average priority is 2.81 and average allowed delay is 1.54 seconds. It is therefore interpreted as high priority and urgent, though its average delay is slightly longer than the Medical group's.

CHAPTER 6: EXPERIMENTAL RESULTS AND DISCUSSION (CONTINUED)

6.6 What the Results Show

The pipeline produces four interpretable categories aligned with the requested data types.

The reported group averages distinguish routine requests from high-priority medical and rescue requests.

The three model comparison rows contain almost identical scores.

The current results support the usefulness of organizing requests, but do not prove that DEC outperforms simpler clustering methods.

The cluster labels are descriptive summaries, not guaranteed ground-truth categories for every individual record.

6.7 Threats to Validity

The project repository identifies all 1,000 records as synthetic. These profiles describe generated examples and cannot establish how real disaster-affected users behave. Scores also depend on feature scaling, model settings, and initialization. Future validation should include repeated runs, sensitivity checks, expert review of real-data examples, and downstream tests in a UAV network simulator.

6.8 Suggested Additional Evaluation

Repeat each model with multiple random seeds and report mean and variation of metrics.

Report the number of records assigned to each cluster and any DBSCAN noise points.

Compare clusters against expert-reviewed examples where reliable labels are available.

Test whether group assignments remain stable when input features are slightly changed.

Measure downstream network-level outcomes in a simulator before making claims about communication performance.

CHAPTER 6: VISUAL OUTPUTS AND RESULT ANALYSIS

Figures 6.1-6.3 show algorithm assignments in the coordinate plane. Figures 6.4 and 6.8 use saved repository metric files; Figures 6.5-6.7 use the saved DEC cluster summary. These figures share a consistent source with the report's result tables. Internal clustering scores do not test real UAV network performance.

Figure 6.1: K-Means User Grouping

The plot displays four visibly separated groups in the coordinate plane. Two groups are concentrated toward the left side and two toward the right side, with upper and lower groupings.

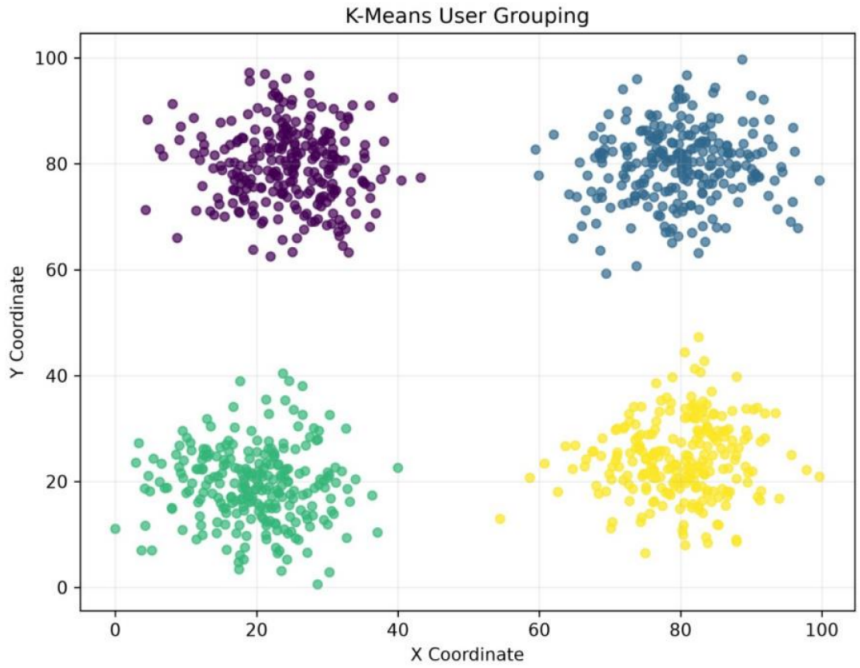


Figure 6.2: DBSCAN User Grouping

The DBSCAN visualization shows four dense regions in a similar coordinate layout. The plot provides a visual comparison with K-Means for these supplied points.

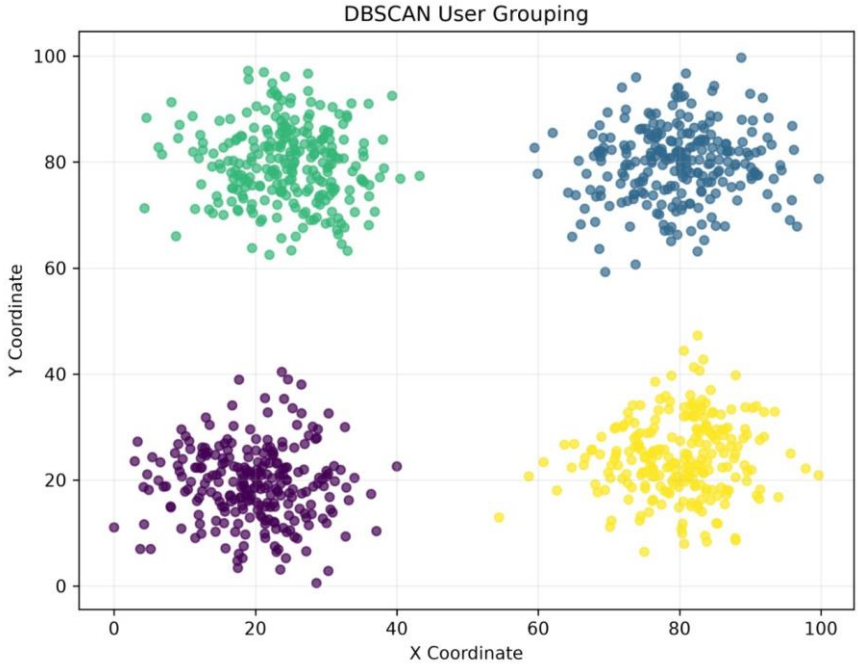

DBSCAN User Grouping
Y Coordinate
X Coordinate
0
20
40
60
80
100

Figure 6.3: DEC Semantic User Grouping

The DEC visualization shows the learned cluster labels for the displayed sample. Compared with the first two plots, some groups overlap more in this coordinate projection; this visual alone does not establish overall model quality.

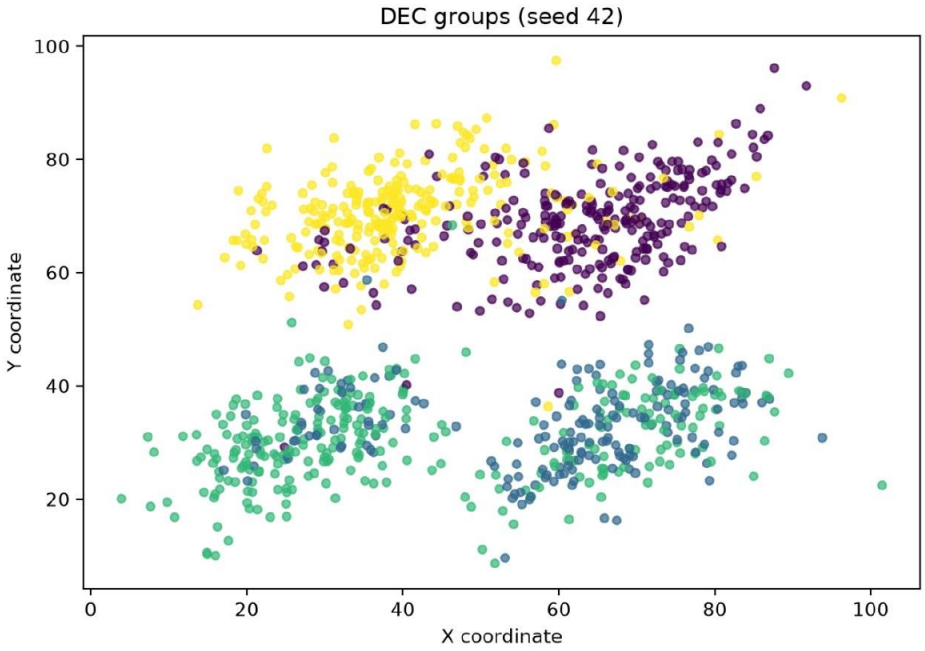


Figure 6.4: Clustering Metric Comparison

The scores come from the repository's saved model_comparison.csv. Silhouette and Calinski-Harabasz are higher-is-better; Davies-Bouldin is lower-is-better. The methods are effectively tied on this synthetic dataset.

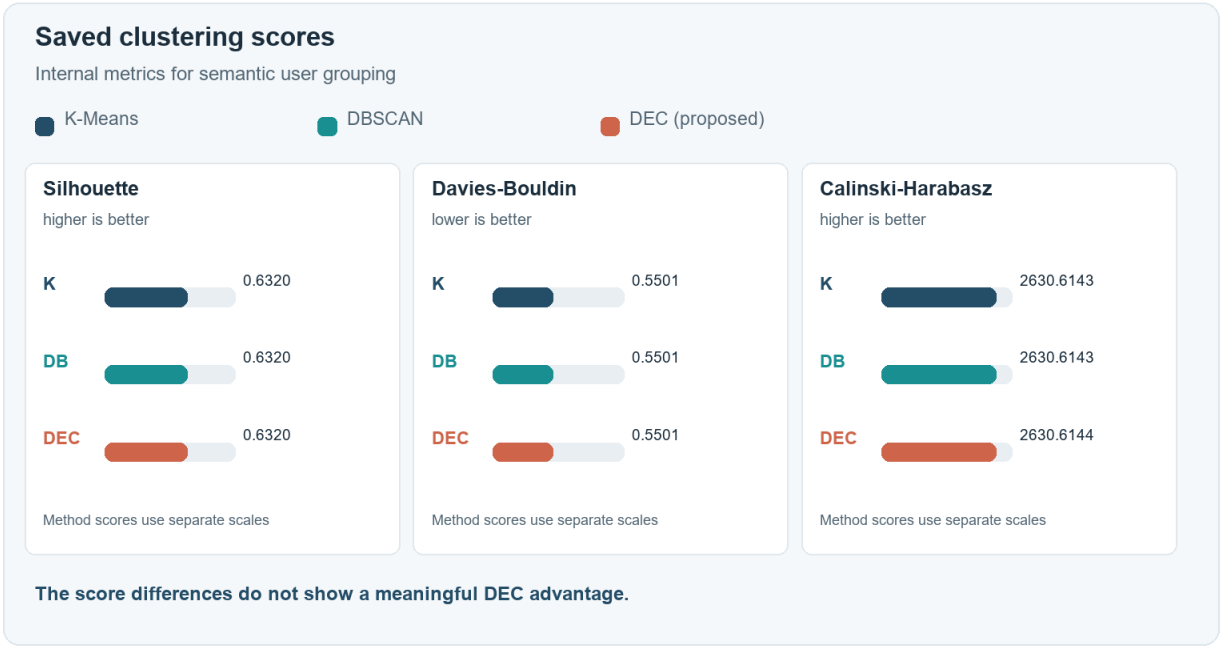


Figure 6.5: DEC Semantic User Group Counts

Counts from results/dec_cluster_analysis.csv are 253 General, 253 Medical, 256 Location, and 238 Rescue. Location is largest; Rescue is smallest. The four groups are close in size.

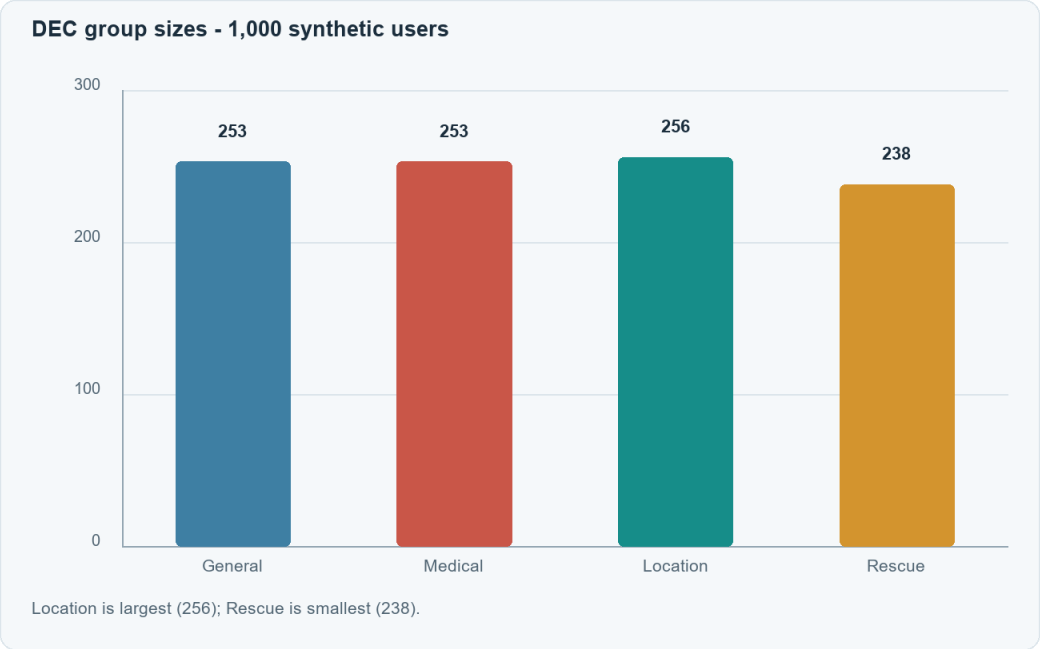


Figure 6.6: Average Emergency Priority by DEC Cluster

Medical and Rescue have nearly identical mean priority (2.81); General is lowest at 1.21. These averages describe the synthetic input records.

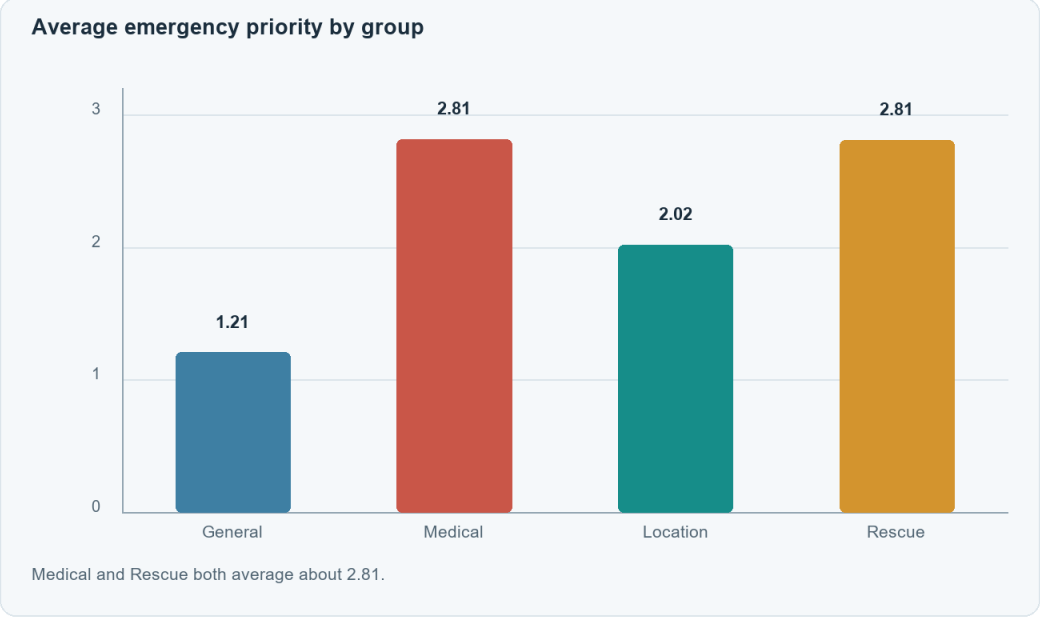


Figure 6.7: Average Delay Requirement by DEC Cluster

Mean delay is lowest for Medical (0.98 s), then Rescue (1.54 s), Location (3.00 s), and General (5.95 s). Lower requirements mean less delay tolerance in this dataset.

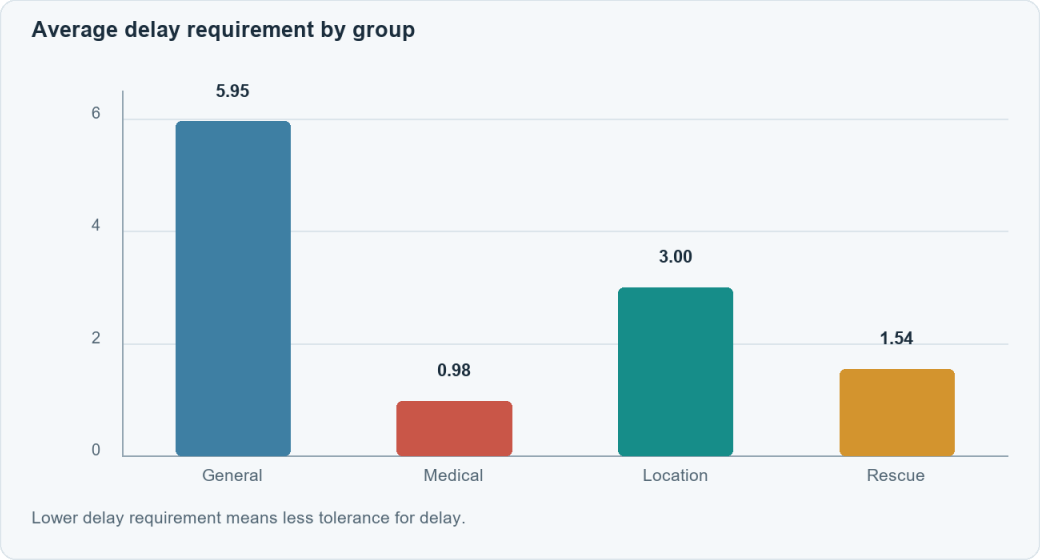
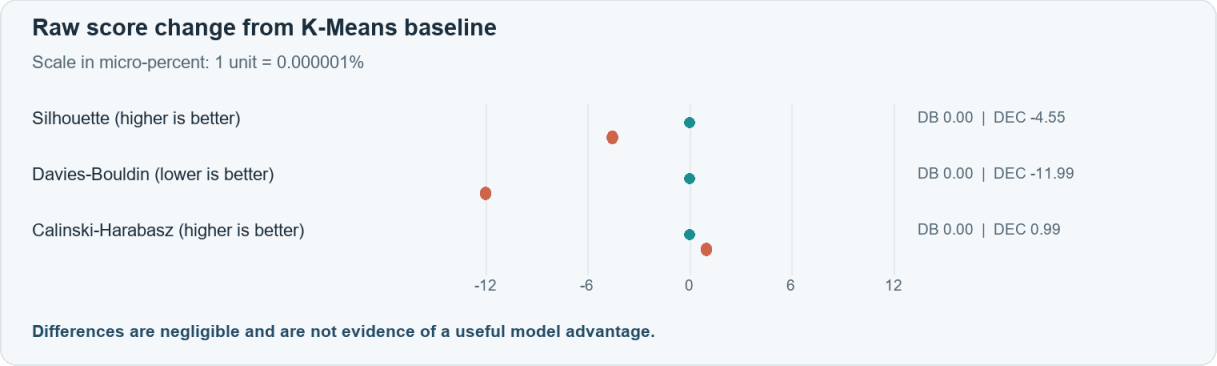


Figure 6.8: Metric Score Differences from K-Means

This chart shows raw score changes from the K-Means baseline at a micro-percent scale. The largest change is only 0.000012%; this is descriptive and does not show a useful model advantage.



Note on Consistency of Results

Figures 6.4-6.8 now use the repository's saved comparison and DEC profile CSVs. The scores are essentially tied; decimal-level differences do not establish a meaningful or statistically significant DEC advantage. All inputs are synthetic, and no operational user study or live UAV-network test is reported.

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Conclusion

This project develops an unsupervised machine learning workflow for grouping disaster-affected users according to communication needs, with the goal of supporting future UAV-assisted communication. The prepared input contains 1,000 user records represented by ten features: six standardized numerical features and four one-hot encoded requested-data categories. K-Means and DBSCAN serve as baseline methods, while DEC learns a five-dimensional representation through an autoencoder and refines cluster assignments.

The project review reports four human-interpretable groups: General, Medical, Location, and Rescue. The average priority and delay values indicate that Medical and Rescue requests are generally more urgent in the current dataset, while General requests allow more delay. The workflow also includes model comparison, cluster profiling, new-user prediction, reporting outputs, and a Streamlit dashboard.

The reported evaluation scores are almost identical across all three methods. As a result, the current experiment does not establish that DEC is significantly better than K-Means or DBSCAN. The project nevertheless demonstrates an end-to-end way to prepare communication features, apply clustering, and summarize groups for potential downstream use. Real-world usefulness remains to be tested using representative data and a UAV network simulator.

7.2 Future Work

Test the models on larger and more varied disaster communication datasets.

Tune DEC architecture, training settings, and clustering parameters systematically.

Investigate the near-identical metric values and verify all scores from saved model outputs.

Measure cluster stability across random seeds and changing input conditions.

Validate group interpretations with disaster-response and communication-network experts.

Connect the group output to a UAV network simulator to evaluate latency, coverage, capacity, and priority-aware resource allocation.

Add privacy, security, and human oversight requirements before any operational deployment.

REFERENCES

Machine Learning Project Review 2 presentation supplied with this report: “Deep Embedded Clustering-Based Semantic User Grouping for UAV-Assisted Disaster Communication,” Machine Learning (25SC2107E), Academic Year 2026–2027.

Project source repository: https://github.com/chandu235363/semantic-uav-disaster-communication

MacQueen, J. (1967). Some Methods for Classification and Analysis of Multivariate Observations. Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability.

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. Proceedings of the Second International Conference on Knowledge Discovery and Data Mining.

Xie, J., Girshick, R., & Farhadi, A. (2016). Unsupervised Deep Embedding for Clustering Analysis. Proceedings of the 33rd International Conference on Machine Learning.

Rousseeuw, P. J. (1987). Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis. Journal of Computational and Applied Mathematics, 20, 53–65.

Scikit-learn documentation, clustering and clustering performance evaluation: https://scikit-learn.org/stable/modules/clustering.html

TensorFlow documentation: https://www.tensorflow.org/ and Keras documentation: https://keras.io/

Streamlit documentation: https://docs.streamlit.io/

APPENDIX A: PROJECT SUMMARY

Item	Project detail
Project title	Deep Embedded Clustering-Based Semantic User Grouping for UAV-Assisted Disaster Communication
Learning type	Unsupervised machine learning
Dataset size	1,000 users
Raw columns	8
Final model input	10 features (6 scaled numerical + 4 one-hot columns)
Models	K-Means, DBSCAN, DEC
DEC latent size	5 features
Output groups	General, Medical, Location, Rescue
Software	Python 3.10, TensorFlow/Keras, Scikit-learn, Streamlit
Main future direction	Connect groups to a UAV network simulator

APPENDIX B: ABBREVIATIONS

Abbreviation	Meaning
DEC	Deep Embedded Clustering
UAV	Unmanned Aerial Vehicle
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
KL divergence	Kullback–Leibler divergence
ML	Machine Learning
AI	Artificial Intelligence
API	Application Programming Interface